

```

class InvalidInputError(Exception):
    """Exception raised for invalid input values."""

    def __init__(self, message="Invalid input provided.", value=None):
        self.message = message
        self.value = value
        super().__init__(self.message)

    def __str__(self):
        if self.value is not None:
            return f"{self.message} Received: {self.value}"
        return self.message

def process_data(data):
    if not isinstance(data, int) or data < 0:
        raise InvalidInputError("Data must be a non-negative integer.", value=c
# Further processing of valid data
    return data * 2

try:
    result = process_data("abc")
    print(result)
except InvalidInputError as e:
    print(f"Error: {e}")

try:
    result = process_data(-5)
    print(result)
except InvalidInputError as e:
    print(f"Error: {e}")

try:
    result = process_data(10)
    print(result)
except InvalidInputError as e:
    print(f"Error: {e}")

Error: Data must be a non-negative integer. Received: abc
Error: Data must be a non-negative integer. Received: -5
20

```

Benefits of Custom Exceptions:

Clarity and Specificity: Custom exceptions provide meaningful error messages tailored to specific application contexts, improving code readability and understanding.

Granular Error Handling: They enable more precise error handling, allowing you to catch and address specific error conditions individually.

Improved Maintainability: By encapsulating specific error logic within custom exception classes, you can centralize error handling and make your code easier to maintain.

Reusability: Custom exceptions can be reused across different parts of your application or even in other projects, promoting consistency in error management.